

```

/*
=====
=====
    GAMEWIN – APP.JS – BLOC 1/5
    Base : données, navigation, profil,
    récompenses, admin
    (colle les blocs 1 → 5 dans cet ordre,
    sans rien enlever)
=====
===== */
const $ = id =>
document.getElementById(id);
const MULT = [1, 1.5, 2.2]; //
multiplicateur de points : facile / moyen /
difficile
const shuffle = a => { for (let i =
a.length - 1; i > 0; i--) { const j =
Math.floor(Math.random() * (i + 1)); [a[i],
a[j]] = [a[j], a[i]]; } return a; };
let stopper = null; // arrête le jeu 3D /
chrono en cours
function stopAll() { if (stopper) { try {
stopper(); } catch (e) {} stopper = null; }
}

const defaultGames = [
  { name: "Quiz Culture Générale", icon:
  "🧠", points: 50, type: "quiz" },
  { name: "Puzzle", icon: "🧩", points: 40,
  type: "puzzle" },

```

```

    { name: "Défi Rapide", icon: "⚡",
      points: 60, type: "speed" },
    { name: "Tir de précision", icon: "🎯",
      points: 50, type: "aim" },
    { name: "Échecs", icon: "♟️", points: 70,
      type: "chess" },
    { name: "Tournoi", icon: "🏆", points:
      100, type: "tournament" },
    { name: "Devine le nombre", icon: "🎲",
      points: 50, type: "number" },
    { name: "Ludo", icon: "🎲", points: 100,
      type: "ludo" }
  ];
  const typeByName = {};
  defaultGames.forEach(g =>
    typeByName[g.name] = g.type);

  const defaultRewards = [
    { name: "Badge Champion", cost: 500,
      icon: "🏆" },
    { name: "Carte cadeau", cost: 2000, icon:
      "🎁" },
    { name: "Accessoire gaming", cost: 5000,
      icon: "🎧" }
  ];

  function loadJSON(key, fallback) {
    try { const v =
      JSON.parse(localStorage.getItem(key));
      return v === null ? fallback : v; }
    catch (e) { return fallback; }
  }

  let games = loadJSON("games", null);

```

```
if (!Array.isArray(games) || games.length
=== 0) games = defaultGames.slice();
// anciens jeux enregistrés en "soon" → on
les active
games.forEach(g => { if ((!g.type || g.type
=== "soon") && typeByName[g.name]) g.type =
typeByName[g.name]; });

let rewards = loadJSON("rewards",
defaultRewards.slice());
if (!Array.isArray(rewards) ||
rewards.length === 0) rewards =
defaultRewards.slice();
let player = loadJSON("player", { name:
"Visiteur", points: 0 });
if (!player || typeof player !== "object")
player = { name: "Visiteur", points: 0 };
let rewardRequests =
loadJSON("rewardRequests", []);
if (!Array.isArray(rewardRequests))
rewardRequests = [];

function saveData() {
  localStorage.setItem("games",
JSON.stringify(games));
  localStorage.setItem("rewards",
JSON.stringify(rewards));
  localStorage.setItem("player",
JSON.stringify(player));
  localStorage.setItem("rewardRequests",
JSON.stringify(rewardRequests));
  render();
}
function addPoints(n) { player.points += n;
```

```

saveData(); }

/* ----- Mises (points virtuels) &
célébration ----- */
let activeBet = null;
function startWithBet(fn) {
  activeBet = null;
  if (player.name === "Visiteur") {
    alert("Inscris-toi (bouton en haut) pour
pouvoir miser des points. Tu joues sans
mise."); return window[fn](); }
  const opts = [0, 10, 25, 50,
100].filter(x => x === 0 || x <=
player.points);
  const b = opts.map(x => `<button
class="primary"
style="display:block;width:100%;margin:8px
0" onclick="placeBet(${x}, '${fn}')">${x} ?
"🎰 Miser " + x + " points" : "Jouer sans
miser"</button>`).join("");
  ensurePage("betPage", `<div
class="quizbox"><h2>💰 Ta mise</h2><p>Tes
points : <b>${player.points}</b></p>
<p class="muted">Victoire : tu
récupères ta mise ×2. Défaite : la mise est
perdue. Match nul : mise rendue. Dans tous
les cas, bonus fidélité de 10 % de la mise.
</p>${b}
<button class="primary"
onclick="showPage('games')">↩
Jeux</button></div>`);
  showPage("betPage");
}
function placeBet(x, fn) {

```

```

    activeBet = null;
    if (x > 0) {
        if (player.points < x) { alert("Pas
assez de points."); return; }
        player.points -= x; activeBet = {
stake: x }; saveData();
    }
    window[fn]();
}
// won : true | false | "draw"
function gameOver(won, info) {
    let msg = "";
    if (activeBet) {
        const s = activeBet.stake, bonus =
Math.max(1, Math.round(s * 0.1));
        let gain = bonus;
        if (won === true) { gain += s * 2; msg
= `🎰 Mise ${s} gagnée : +${s * 2} points`;
        }

        else if (won === "draw") { gain += s;
msg = `🎰 Mise ${s} rendue`; }
        else msg = `🎰 Mise ${s} perdue`;
        msg += `<br>🎁 Bonus fidélité :
+${bonus} points`;
        player.points += gain; activeBet =
null; saveData();
    }
    celebrate(won, (info || "") + (msg ? "<br>" + msg : ""));
}
function celebrate(won, html) {
    if (!$("gwCss")) { const st =
document.createElement("style"); st.id =
"gwCss"; st.textContent = "@keyframes

```

```

gwfall{to{transform:translateY(110vh)
rotate(540deg)}}@keyframes
gwpop{from{transform:scale(.4);opacity:0}to
{transform:scale(1);opacity:1}}";
document.head.appendChild(st); }
    const old = $("gwCeleb"); if (old)
old.remove();
    const o = document.createElement("div");
o.id = "gwCeleb";
    o.style.cssText =
"position:fixed;inset:0;z-
index:9999;background:#000b;display:flex;al
ign-items:center;justify-
content:center;overflow:hidden";
    let c = ""; const E = ["🎉", "🌟", "🏆",
"🌟", "🏆"];
    for (let i = 0; i < 55; i++) c += `<span
style="position:absolute;top:-40px;left:${M
ath.random() * 100}%;font-size:${16 +
Math.random() * 22}px;animation:gwfall ${2
+ Math.random() * 2.5}s linear
${Math.random() * 1.2}s forwards">${E[i %
5]}</span>`;
    o.innerHTML = c + `<div
style="position:relative;background:#0d1d36
;border:2px solid #ffd21c;border-
radius:20px;padding:24px;text-
align:center;max-
width:320px;width:86%;animation:gwpop .4s">
<div style="font-size:56px">${won === true
? "🏆" : "🎉"}</div><h2 style="margin:6px
0">${won === true ? "Victoire !" : won ===
"draw" ? "Match nul !" : "Partie terminée
!"}</h2><p>${html}</p><button

```

```

class="primary"
onclick="document.getElementById('gwCeleb')
.remove()">Continuer</button></div>`;
    document.body.appendChild(o);
}

/* ----- Pages dynamiques & choix de
difficulté ----- */
function ensurePage(id, html) {
    let p = $(id);
    if (!p) { p =
document.createElement("section"); p.id =
id; p.className = "page";
document.querySelector("main").appendChild(
p); }
    p.innerHTML = html;
    return p;
}
function pickLevel(title, cb) {
    window._lvCb = cb;
    const b = (l, t) => `<button
class="primary"
style="display:block;width:100%;margin:8px
0" onclick="_lv(${l})">${t}</button>`;
    ensurePage("levelPage", `<div
class="quizbox"><h2>${title}</h2><p>Choisis
la difficulté :</p>${b(0, "●
Facile")}${b(1, "● Moyen (points
×1,5)")}${b(2, "● Difficile (points
×2,2)")}<button class="primary"
style="margin-top:10px"
onclick="showPage('games')">↵
Jeux</button></div>`);
    showPage("levelPage");
}

```

```

}
function _lv(l) { if (window._lvCb)
window._lvCb(l); }

/* ----- Navigation ----- */
function renderLudo() {} //
remplacé par ludo.js
function startLudo() {
showPage("ludoGame"); }
function showPage(id) {
    stopAll();

document.querySelectorAll(".page").forEach(
p => p.classList.remove("active"));
    const page = $(id);
    if (page) page.classList.add("active");
    if (id === "ranking") renderRanking();
    if (id === "rewards") renderRewards();
    if (id === "admin") renderAdmin();
    if (id === "ludoGame") renderLudo();
    window.scrollTo(0, 0);
}

function render() {
    const ua = $("userArea");
    if (ua) ua.innerHTML = `

```

```

player.points);
    set("aGames", games.length);
set("aPlayers", "1"); set("aPoints",
player.points);
    renderGames(); renderRewards();
renderRanking(); renderAdmin();
}

const STARTERS = { quiz: "startQuiz",
number: "startNumberGame", ludo:
"startLudo", puzzle: "startPuzzle", speed:
"startSpeed", aim: "startAim", chess:
"startChess", tournament: "startTournament"
};
function renderGames() {
    const grid = $("gameGrid");
    if (!grid) return;
    grid.innerHTML = games.map(g => {
        const fn = STARTERS[g.type];
        const btn = fn ? `<button
class="primary"
onclick="startWithBet('${fn}')">Jouer</butt
on>` : `<button class="primary"
onclick="comingSoon()">Bientôt</button>`;
        return `<div class="game"><div
class="icon">${g.icon}</div><h3>${g.name}
</h3><p class="muted">Joue et gagne jusqu'à
${g.points} points.</p>${btn}</div>`;
    }).join("");
}
function comingSoon() { alert("🎮 Ce jeu
sera bientôt disponible !"); }

/* ----- Profil ----- */

```

```

function openLogin() {
  const m = $("loginModal"); if (m)
m.classList.remove("hidden");
  const i = $("username");
  if (i) { i.value = player.name ===
"Visiteur" ? "" : player.name;
setTimeout(() => i.focus(), 100); }
}
function closeLogin() { const m =
$("loginModal"); if (m)
m.classList.add("hidden"); }
function login() {
  const i = $("username"); if (!i) return;
  const name = i.value.trim();
  if (name.length < 2) { alert("Entre un
pseudo d'au moins 2 caractères."); return;
}
  player.name = name; saveData();
closeLogin(); alert("Bienvenue " + name + "
! 🎮");
}

/* ----- Récompenses ----- */
function renderRewards() {
  const grid = $("rewardGrid"); if (!grid)
return;
  grid.innerHTML = rewards.map((r, i) =>
`<div class="game"><div
class="icon">${r.icon} || "🎁"</div>
<h3>${r.name}</h3><p class="muted">Coût :
<strong>${r.cost}</strong> points</p>
<button class="primary"
onclick="claimReward(${i})">Réclamer</butto
n></div>`).join("");

```

```

}
function claimReward(i) {
  const r = rewards[i]; if (!r) return;
  if (player.points < r.cost) { alert("❌
Tu n'as pas assez de points."); return; }
  if (!confirm("Réclamer " + r.name + "
pour " + r.cost + " points ?")) return;
  player.points -= r.cost;
  rewardRequests.push({ id: Date.now(),
player: player.name, reward: r.name, cost:
r.cost, status: "pending" });
  saveData(); alert("✅ Demande envoyée à
l'administrateur.");
}
function renderRanking() {
  const b = $("rankingBody");
  if (b) b.innerHTML = `<tr><td>1</td>
<td>${player.name}</td><td>${player.points}
</td></tr>`;
}

/* ----- Admin ----- */
function renderAdmin() {
  const box = $("rewardRequests"); if
(!box) return;
  if (!rewardRequests.length) {
box.innerHTML = `<p class="muted">Aucune
demande de récompense.</p>`; return; }
  box.innerHTML = rewardRequests.map((q, i)
=> {
    const st = q.status === "approved" ?
"✅ Validée" : q.status === "rejected" ?
"❌ Refusée" : "⌚ En attente";
    const act = q.status === "pending" ?

```

```

`<div style="margin-top:10px"><button
class="primary"
onclick="approveReward(${i})">✓
Valider</button> <button class="primary"
onclick="rejectReward(${i})">✗
Refuser</button></div>` : "";
    return `<div class="panel"
style="margin-bottom:12px">
<strong>${q.reward}</strong><p>Joueur :
${q.player}</p><p>Coût : ${q.cost}
points</p><p>${st}</p>${act}</div>`;
  }).join("");
}
function approveReward(i) {
  const q = rewardRequests[i]; if (!q ||
q.status !== "pending") return;
  q.status = "approved"; saveData();
  alert("Récompense validée. ✓");
}
function rejectReward(i) {
  const q = rewardRequests[i]; if (!q ||
q.status !== "pending") return;
  q.status = "rejected"; player.points +=
Number(q.cost) || 0; saveData();
  alert("Récompense refusée. Les points ont
été remboursés. ✓");
}
function addGame() {
  const n = $("newGameName"), p =
$("newGamePoints"); if (!n || !p) return;
  const name = n.value.trim(), pts =
Number(p.value);
  if (!name) { alert("Entre le nom du
jeu."); return; }

```

```

    if (!Number.isFinite(pts) || pts <= 0) {
    alert("Entre un nombre de points valide.");
    return; }

    games.push({ name, icon: "🎮", points:
pts, type: "soon" });
    n.value = ""; p.value = ""; saveData();
    alert("🎮 Jeu ajouté avec succès.");
  }
  function addReward() {
    const n = $("newRewardName"), c =
$("newRewardCost"); if (!n || !c) return;
    const name = n.value.trim(), cost =
Number(c.value);
    if (!name) { alert("Entre le nom de la
récompense."); return; }
    if (!Number.isFinite(cost) || cost <= 0)
{ alert("Entre un coût valide."); return; }
    rewards.push({ name, cost, icon: "🎁" });
    n.value = ""; c.value = ""; saveData();
    alert("🎁 Récompense ajoutée avec
succès.");
  }
  /* ===== FIN BLOC 1 ===== */
  /*
=====
=====
    GAMEWIN – APP.JS – BLOC 2/5
    Quiz (3 niveaux, chrono) + Devine le
nombre (3 niveaux)
=====
===== */
const QBANK = [
  { q: "Quelle est la capitale du Burkina
Faso ?", a: ["Bobo-Dioulasso",

```

```
"Ouagadougou", "Koudougou", "Banfora"], c:
1 },
  { q: "Combien font  $7 \times 8$  ?", a: ["54",
"56", "64", "48"], c: 1 },
  { q: "Quelle planète est surnommée la
planète rouge ?", a: ["Mars", "Vénus",
"Jupiter", "Mercure"], c: 0 },
  { q: "Combien y a-t-il de continents ?",
a: ["5", "6", "7", "8"], c: 2 },
  { q: "Quel est le plus grand océan du
monde ?", a: ["Atlantique", "Indien",
"Arctique", "Pacifique"], c: 3 },
  { q: "Quel est le plus long fleuve
d'Afrique ?", a: ["Niger", "Nil", "Congo",
"Zambèze"], c: 1 },
  { q: "Quel est le symbole chimique de
l'or ?", a: ["Ag", "Au", "Or", "Fe"], c: 1
},
  { q: "En quelle année la Haute-Volta est-
elle devenue indépendante ?", a: ["1958",
"1960", "1962", "1966"], c: 1 },
  { q: "Quelle est la racine carrée de 144
?", a: ["11", "12", "14", "16"], c: 1 },
  { q: "Quel est le plus petit nombre
premier ?", a: ["0", "1", "2", "3"], c: 2
},
  { q: "Qui a écrit « Les Misérables » ?",
a: ["Zola", "Victor Hugo", "Balzac",
"Camus"], c: 1 },
  { q: "Combien d'os a un adulte ?", a:
["186", "206", "226", "256"], c: 1 },
  { q: "Quel est le plus grand pays du
monde ?", a: ["Canada", "Chine", "Russie",
"États-Unis"], c: 2 },
```

```
{ q: "Quelle est la capitale du Japon ?",  
a: ["Kyoto", "Osaka", "Tokyo", "Séoul"], c:  
2 },  
{ q: "Quel gaz est le plus abondant dans  
l'atmosphère ?", a: ["Oxygène", "Azote",  
"CO2", "Argon"], c: 1 },  
{ q: "Vitesse approximative de la lumière  
?", a: ["3 000 km/s", "30 000 km/s", "300  
000 km/s", "3 000 000 km/s"], c: 2 },  
{ q: "Quel est le plus haut sommet du  
monde ?", a: ["K2", "Everest",  
"Kilimandjaro", "Mont Blanc"], c: 1 },  
{ q: "Combien de côtés a un hexagone ?",  
a: ["5", "6", "7", "8"], c: 1 },  
{ q: "Quelle monnaie est utilisée au  
Burkina Faso ?", a: ["Euro", "Dollar",  
"Franc CFA", "Naira"], c: 2 },  
{ q: "Quelle est la formule chimique de  
l'eau ?", a: ["CO2", "H2O", "O2", "NaCl"],  
c: 1 },  
{ q: "Quel est le résultat de  $17 \times 13$  ?",  
a: ["211", "221", "231", "241"], c: 1 },  
{ q: "Quel élément a pour numéro atomique  
6 ?", a: ["Azote", "Oxygène", "Carbone",  
"Hélium"], c: 2 },  
{ q: "En quelle année est tombé le mur de  
Berlin ?", a: ["1987", "1989", "1991",  
"1993"], c: 1 },  
{ q: "Quelle est la capitale de  
l'Australie ?", a: ["Sydney", "Melbourne",  
"Canberra", "Perth"], c: 2 },  
{ q: "Combien font 2 puissance 10 ?", a:  
["512", "1000", "1024", "2048"], c: 2 },  
{ q: "Quel est le plus grand désert chaud
```

```

du monde ?", a: ["Gobi", "Sahara",
"Kalahari", "Atacama"], c: 1 },
  { q: "Qui a peint la Joconde ?", a:
["Michel-Ange", "Raphaël", "Léonard de
Vinci", "Botticelli"], c: 2 },
  { q: "Quelle est la capitale du Mali ?",
a: ["Bamako", "Niamey", "Dakar",
"Conakry"], c: 0 },
  { q: "Quel organe produit l'insuline ?",
a: ["Foie", "Pancréas", "Rein", "Rate"], c:
1 },
  { q: "Combien de joueurs y a-t-il dans
une équipe de football sur le terrain ?",
a: ["9", "10", "11", "12"], c: 2 },
  { q: "Quel est le nombre  $\pi$  arrondi à 2
décimales ?", a: ["3,12", "3,14", "3,16",
"3,18"], c: 1 },
  { q: "Quel pays a pour capitale
Ouagadougou ?", a: ["Mali", "Niger",
"Burkina Faso", "Ghana"], c: 2 }
];

let qz = null;
function startQuiz() {
  pickLevel("🧠 Quiz Culture Générale", lv
=> {
    const n = [10, 14, 20][lv];
    qz = { lv, i: 0, s: 0, list:
shuffle(QBANK.slice()).slice(0, n).map(q =>
({ q: q.q, o: shuffle(q.a.map((t, k) => ({
t, ok: k === q.c }))) }))) });
    quizNext();
  });
}

```

```

function quizNext() {
  showPage("quiz");
  const q = qz.list[qz.i];
  $("quizMeta").textContent = `Question
  ${qz.i + 1} / ${qz.list.length} - Score
  ${qz.s}`;
  $("question").textContent = q.q;
  $("quizResult").textContent = "";
  $("answers").innerHTML = q.o.map((o, k)
=> `

```

```

    if (qz.i >= qz.list.length) {
        addPoints(qz.s);
        gameOver(qz.s >=
Math.round(qz.list.length * 10 *
MULT[qz.lv] * 0.6), "Score : " + qz.s + "
points");
        $("quizResult").innerHTML = "🎉 Quiz
terminé ! +" + qz.s + " points";
        $("answers").innerHTML = `<button
class="primary"
onclick="startWithBet('startQuiz')">🔄
Rejouer</button> <button class="primary"
onclick="showPage('games')">Retour aux
jeux</button>`;
        return;
    }
    quizNext();
}

/* ----- Devine le nombre -----
*/
let nb = null;
function startNumberGame() {
    pickLevel("1 2 Devine le nombre", lv => {
        const max = [100, 500, 1000][lv], tries
= [8, 9, 10][lv];
        nb = { lv, max, left: tries, n: 1 +
Math.floor(Math.random() * max), over:
false };
        const i = $("numberGuess"); i.value =
""; i.min = 1; i.max = max;
        $("numberResult").textContent = `J'ai
choisi un nombre entre 1 et ${max}. Tu as
${tries} essais.`;

```

```

        showPage("numberGame");
    });
}
function guessNumber() {
    if (!nb || nb.over) return;
    const g = Number($("#numberGuess").value),
    r = $("#numberResult");
    if (!Number.isInteger(g) || g < 1 || g >
nb.max) { r.textContent = "Entre un nombre
entre 1 et " + nb.max + "."; return; }
    nb.left--;
    if (g === nb.n) {
        const pts = Math.round((20 + nb.left *
5) * MULT[nb.lv]);
        nb.over = true; addPoints(pts);
gameOver(true, "Trouvé : " + nb.n);
        r.innerHTML = `🎉 Bravo ! C'était
${nb.n}. +${pts} points !`;
    } else if (nb.left <= 0) {
        nb.over = true; r.textContent = `💣
Perdu ! Le nombre était ${nb.n}.`;
gameOver(false, "Le nombre était " + nb.n);
    } else {
        r.textContent = (g < nb.n ? "⬆️ Plus
grand !" : "⬇️ Plus petit !") + `
(${nb.left} essais restants)`;
    }
    $("#numberGuess").value = "";
}
/* ===== FIN BLOC 2 ===== */
/*
=====
=====

```

```

Défi Rapide (calcul mental, carte 3D) +
Tournoi (3 manches)
=====
===== */
let sp = null;
function spQuestion(lv) {
  const r = (a, b) => a +
Math.floor(Math.random() * (b - a + 1));
  let a, b, c;
  if (lv === 0) { a = r(3, 12); b = r(3,
12); c = r(5, 60); return Math.random() <
.5 ? [`${a} × {b}`, a * b] : [`${a} × {b}
+ {c}`, a * b + c]; }
  if (lv === 1) { a = r(11, 25); b = r(3,
12); c = r(10, 99); return Math.random() <
.5 ? [`${a} × {b}`, a * b] : [`${a} × {b}
- {c}`, a * b - c]; }
  a = r(12, 29); b = r(6, 19); c = r(20,
150);
  const k = r(0, 2);
  return k === 0 ? [`${a} × {b} + {c}`, a
* b + c] : k === 1 ? [`${c} + {a}) ×
{b}`, (c + a) * b] : [`${a} × {b} -
{c}`, a * b - c];
}
// moteur commun : joue une manche puis
appelle cb(score)
function speedRun(title, lv, secs, cb) {
  ensurePage("speedPage", `
```

```

style="transition:transform .4s;transform-
style:preserve-3d;background:linear-
gradient(135deg,#1c3f78,#2b2a7a);border:2px
solid #ffd21c;border-
radius:18px;padding:30px 10px;text-
align:center;font-size:44px;font-
weight:800;box-shadow:0 14px 28px #0008">
<span id="spQ"></span></div></div>
    <input id="spIn" type="number"
inputmode="numeric" placeholder="Ta
réponse"><button class="primary"
onclick="spOk()">OK</button>
    <p id="spMsg" class="muted"></p><button
class="primary"
onclick="showPage('games')">←
Jeux</button></div>`);
    showPage("speedPage");
    sp = { lv, t: secs, s: 0, ans: 0, cb,
rot: 0, over: false };
    spNext();
    const iv = setInterval(() => {
        sp.t--; spMeta();
        if (sp.t <= 0) { clearInterval(iv);
sp.over = true; const f = sp.cb; f(sp.s); }
    }, 1000);
    stopper = () => { clearInterval(iv); if
(sp) sp.over = true; };
    $("spIn").onkeydown = e => { if (e.key
=== "Enter") spOk(); };
    setTimeout(() => $("spIn").focus(), 150);
}
function spMeta() { $("spMeta").textContent
= `🕒 ${Math.max(0, sp.t)} s — Score :
${sp.s}`; }

```

```

function spNext() {
  const [q, a] = spQuestion(sp.lv);
  sp.ans = a; $("spQ").textContent = q + "
= ?";
  sp.rot += 360;
$("spCard").style.transform =
`rotateX(${sp.rot}deg)`;
  $("spIn").value = ""; spMeta();
}
function spOk() {
  if (!sp || sp.over) return;
  const v = $("spIn").value;
  if (v === "") return;
  if (Number(v) === sp.ans) { sp.s++;
$("spMsg").textContent = "✅ Bien joué !";
}
  else { sp.t = Math.max(0, sp.t - 3);
$("spMsg").textContent = `❌ C'était
${sp.ans} (-3 s)`; }
  spNext();
}
function startSpeed() {
  pickLevel("⚡ Défi Rapide", lv => {
    const secs = [30, 25, 20][lv];
    speedRun("⚡ Défi Rapide", lv, secs,
score => {
      const pts = Math.min(Math.round(60 *
MULT[lv]), Math.round(score * 3 *
MULT[lv]));
      addPoints(pts); gameOver(score >= [8,
6, 4][lv], score + " bonnes réponses");
      $("spQ").textContent = `Terminé !`;
      $("spMsg").innerHTML = `🎉 ${score}
bonnes réponses - +${pts} points<br><button

```

```

class="primary"
onclick="startWithBet('startSpeed')">🔄
Rejouer</button>`;
    });
  });
}

/* ----- Tournoi : 3 manches de
difficulté croissante ----- */
let tn = null;
function startTournament() {
  tn = { round: 0, total: 0 };
  tournamentRound();
}
function tournamentRound() {
  const r = tn.round;
  ensurePage("tournPage", `
```

```

        if (tn.round >= 3) {
            const pts = Math.min(100, tn.total *
3);
            addPoints(pts); gameOver(tn.total >=
24, "Score final : " + tn.total);
            ensurePage("tournPage", `
```

```

ee.js/r128/three.min.js";
    s.onload = cb;
    s.onerror = () => { alert("Impossible de
charger la 3D. Vérifie ta connexion
Internet puis réessaie.");
showPage("games"); };
    document.head.appendChild(s);
}
function mk3d(host, aspect) {
    const w = Math.min(host.clientWidth ||
340, 600), h = Math.round(w * aspect);
    const r = new THREE.WebGLRenderer({
antialias: true });
    r.setSize(w, h);
r.setPixelRatio(Math.min(window.devicePixel
Ratio || 1, 2));
    host.innerHTML = "";
host.appendChild(r.domElement);
    r.domElement.style.cssText =
"display:block;margin:auto;border-
radius:16px;touch-action:none";
    const scene = new THREE.Scene();
scene.background = new
THREE.Color(0x0a1730);
    const cam = new
THREE.PerspectiveCamera(50, w / h, 0.1,
100);
    scene.add(new
THREE.AmbientLight(0xffffff, 0.7));
    const dl = new
THREE.DirectionalLight(0xffffff, 0.8);
dl.position.set(5, 10, 6); scene.add(dl);
    const o = { scene, cam, r, w, h, loop:
null };

```

```

    let raf = 0, run = true;
    (function tick(t) { if (!run) return; raf
= requestAnimationFrame(tick); if (o.loop)
o.loop(t / 1000); r.render(scene, cam); })
(0);
    stopper = () => { run = false;
cancelAnimationFrame(raf); r.dispose(); };
    return o;
}
function pick3d(o, e, objs) {
    const rc =
o.r.domElement.getBoundingClientRect();
    const v = new THREE.Vector2(((e.clientX -
rc.left) / rc.width) * 2 - 1, -((e.clientY
- rc.top) / rc.height) * 2 + 1);
    const ray = new THREE.Raycaster();
ray.setFromCamera(v, o.cam);
    return ray.intersectObjects(objs, true);
}

/* ----- Tir de précision 3D -----
- */
function startAim() {
    pickLevel("🎯 Tir de précision 3D", lv =>
loadThree(() => {
        ensurePage("aimPage", `
```

```

Jeux</button></div>`);
    showPage("aimPage");
    const o = mk3d$("aimHost"), 0.85);
    o.cam.position.set(0, 2.5, 7);
o.cam.lookAt(0, 2.5, -8);
    const floor = new THREE.Mesh(new
THREE.PlaneGeometry(60, 60), new
THREE.MeshStandardMaterial({ color:
0x14315a }));
    floor.rotation.x = -Math.PI / 2;
o.scene.add(floor);
    const P = { rad: [0.7, 0.5, 0.35][lv],
spd: [1.2, 2.2, 3.2][lv], life: [2.8, 2.0,
1.4][lv], max: [3, 4, 5][lv] };
    let score = 0, shots = 0, left = 30,
last = 0, over = false, ts = [];
    function spawn(now) {
        const g = new THREE.Group();
        [[1, 0xffffffff, 1], [0.66, 0xe53935,
2], [0.33, 0xffffffff, 3]].forEach(([k, c,
pt], i) => {
            const m = new THREE.Mesh(new
THREE.CylinderGeometry(P.rad * k, P.rad *
k, 0.12 + i * 0.03, 28), new
THREE.MeshStandardMaterial({ color: c }));
            m.rotation.x = Math.PI / 2;
m.userData.pt = pt; g.add(m);
        });
        g.userData = { bx: (Math.random() -
0.5) * 8, ph: Math.random() * 6, born: now
};
        g.position.set(g.userData.bx, 1 +
Math.random() * 4, -7 - Math.random() * 6);
o.scene.add(g); ts.push(g);

```

```

    }
    const t0 = performance.now() / 1000;
    o.loop = t => {
      if (over) return;
      const now = performance.now() / 1000,
      el = now - t0;
      left = Math.max(0, 30 - el);
      $("aimMeta").textContent = `🕒
      ${Math.ceil(left)} s – Score : ${score}`;
      while (ts.length < P.max) spawn(now);
      ts = ts.filter(g => {
        if (now - g.userData.born > P.life)
      { o.scene.remove(g); return false; }
        g.position.x = g.userData.bx +
      Math.sin(g.userData.ph + now * P.spd) *
      1.6;

        const s = 1 - (now -
      g.userData.born) / P.life * 0.35;
      g.scale.set(s, s, s);
        return true;
      });
      if (left <= 0) {
        over = true;
        const pts = Math.min(Math.round(60
      * MULT[lv]), Math.round(score * 1.5 *
      MULT[lv]));
        addPoints(pts); gameOver(score >=
      [20, 22, 25][lv], "Score : " + score);
        $("aimEnd").innerHTML = `<p>🎉
      Score ${score} – +${pts} points</p><button
      class="primary"
      onclick="startWithBet('startAim')">🔄
      Rejouer</button>`;
      }

```

```

    };

    o.r.domElement.addEventListener("pointerdown", e => {
        if (over) return;
        shots++;
        const hit = pick3d(o, e, ts)[0];
        if (hit) {
            let g = hit.object; while (g.parent
            && g.parent !== o.scene) g = g.parent;
            score += hit.object.userData.pt ||
            1;
            o.scene.remove(g); ts = ts.filter(x
            => x !== g);
        }
    });
});
}
/* ===== FIN BLOC 4 ===== */
/*
=====
=====
    GAMEWIN – APP.JS – BLOC 5/5
    Puzzle 3D (taquin) + Échecs 3D (contre
    l'ordinateur) + démarrage
=====
===== */

/* ----- Puzzle 3D ----- */
let pz = null;
function startPuzzle() {
    pickLevel("🍄 Puzzle 3D", lv => {
        const n = [4, 5, 6][lv], b =
        [...Array(n * n).keys()].map(i => i + 1);

```

```

    b[n * n - 1] = 0;
    let e = n * n - 1, prev = -1;
    for (let k = 0; k < n * n * [35, 50,
70][lv]; k++) {
        const r = (e / n) | 0, c = e % n, nb
= [];
        if (r > 0) nb.push(e - n); if (r < n
- 1) nb.push(e + n); if (c > 0) nb.push(e -
1); if (c < n - 1) nb.push(e + 1);
        const opts = nb.filter(x => x !==
prev), m = opts[Math.floor(Math.random() *
opts.length)];
        b[e] = b[m]; b[m] = 0; prev = e; e =
m;
    }
    pz = { n, lv, b, mv: 0, t0: Date.now(),
done: false };
    ensurePage("puzzlePage", `
```

```

onclick="startWithBet('startPuzzle')">↩
Nouveau</button> <button class="primary"
onclick="showPage('games')">←
Jeux</button></div>`);
    showPage("puzzlePage");
    const bd = $("pzBoard"), s = 100 / n;
    bd.innerHTML = [...Array(n * n -
1).keys()].map(i => { const v = i + 1;
    return `<div data-v="${v}"
onclick="pzTap(${v})"
style="position:absolute;width:${s}%;height
:${s}%;padding:2px;box-sizing:border-
box;transition:left .15s,top
.15s;transform-style:preserve-3d"><div
style="width:100%;height:100%;border-
radius:10px;background:linear-
gradient(145deg,#ffd21c,#e0a800);color:#111
;font-weight:800;font-size:${n > 4 ? 20 :
28}px;display:flex;align-
items:center;justify-content:center;box-
shadow:0 7px 0 #8a6500,0 12px 14px
#0008;transform:translateZ(14px)">${v}
</div></div>`; }).join("");
    pzPos();
    const iv = setInterval(pzMeta, 1000);
    stopper = () => clearInterval(iv);
    pzMeta();
    });
}
function pzPos() {
    const { n, b } = pz;
    b.forEach((v, i) => { if (!v) return;
const el =
$("pzBoard").querySelector(`[data-

```

```

v="${v}"]`); el.style.left = (i % n) * 100
/ n + "%"; el.style.top = ((i / n) | 0) *
100 / n + "%"; });
}
function pzMeta() { if (pz && !pz.done)
$("pzMeta").textContent = `Coups : ${pz.mv}
- Temps : ${Math.floor((Date.now() - pz.t0)
/ 1000)} s`; }
function pzTap(v) {
  if (!pz || pz.done) return;
  const { n, b } = pz, i = b.indexOf(v), e
= b.indexOf(0);
  if (Math.abs((i / n | 0) - (e / n | 0)) +
Math.abs(i % n - e % n) !== 1) return;
  b[e] = v; b[i] = 0; pz.mv++; pzPos();
pzMeta();
  if (b.every((x, k) => k === n * n - 1 ? x
=== 0 : x === k + 1)) {
    pz.done = true;
    const base = [30, 45, 60][pz.lv], pts =
Math.max(10, base - Math.floor(Math.max(0,
pz.mv - n * n * 3) / 3));
    addPoints(pts); gameOver(true, "Résolu
en " + pz.mv + " coups");
    $("pzEnd").innerHTML = `<p>🎉 Résolu en
${pz.mv} coups - +${pts} points !</p>`;
  }
}

/* ----- Échecs : logique -----
*/
const CHV = { p: 100, n: 320, b: 330, r:
500, q: 900, k: 20000 };
function chInit() {

```

```

    const b = Array(64).fill(""), back =
    "rnbqkbnr";
    for (let c = 0; c < 8; c++) { b[c] =
    back[c]; b[8 + c] = "p"; b[48 + c] = "P";
    b[56 + c] = back[c].toUpperCase(); }
    return b;
}
function chGen(b, w) {
    const m = [];
    for (let i = 0; i < 64; i++) {
        const p = b[i]; if (!p || (p ===
    p.toUpperCase()) !== w) continue;
        const r = i >> 3, c = i & 7, t =
    p.toLowerCase();
        const add = (rr, cc) => {
            if (rr < 0 || rr > 7 || cc < 0 || cc
    > 7) return false;
            const q = b[rr * 8 + cc];
            if (q && (q === q.toUpperCase()) ===
    w) return false;
            m.push([i, rr * 8 + cc]); return !q;
        };
        if (t === "p") {
            const d = w ? -1 : 1, st = w ? 6 : 1;
            if (r + d >= 0 && r + d < 8 && !b[(r
    + d) * 8 + c]) { m.push([i, (r + d) * 8 +
    c]); if (r === st && !b[(r + 2 * d) * 8 +
    c]) m.push([i, (r + 2 * d) * 8 + c]); }
            for (const dc of [-1, 1]) { const rr
    = r + d, cc = c + dc; if (rr < 0 || rr > 7
    || cc < 0 || cc > 7) continue; const q =
    b[rr * 8 + cc]; if (q && (q ===
    q.toUpperCase()) !== w) m.push([i, rr * 8 +
    cc]); }

```

```

    } else if (t === "n") {
        for (const [a, d] of [[1, 2], [2, 1],
        [-1, 2], [-2, 1], [1, -2], [2, -1], [-1,
        -2], [-2, -1]]) add(r + a, c + d);
    } else if (t === "k") {
        for (let a = -1; a <= 1; a++) for
        (let d = -1; d <= 1; d++) if (a || d) add(r
        + a, c + d);
    } else {
        const dg = [[1, 1], [1, -1], [-1, 1],
        [-1, -1]], st = [[1, 0], [-1, 0], [0, 1],
        [0, -1]];
        const dirs = t === "b" ? dg : t ===
        "r" ? st : dg.concat(st);
        for (const [a, d] of dirs) { let rr =
        r + a, cc = c + d; while (add(rr, cc)) { rr
        += a; cc += d; } }
    }
}
return m;
}
function chDo(b, mv) {
    const n = b.slice(), p = n[mv[0]];
    n[mv[1]] = p; n[mv[0]] = "";
    if (p === "P" && mv[1] < 8) n[mv[1]] =
    "Q";
    if (p === "p" && mv[1] > 55) n[mv[1]] =
    "q";
    return n;
}
function chInCheck(b, w) { const ki =
b.indexOf(w ? "K" : "k"); return ki < 0 ||
chGen(b, !w).some(m => m[1] === ki); }
function chLegal(b, w) { return chGen(b,

```

```

w).filter(m => !chInCheck(chDo(b, m), w));
}
function chEval(b) {
  let s = 0;
  for (let i = 0; i < 64; i++) {
    const p = b[i]; if (!p) continue;
    const t = p.toLowerCase(), w = p ===
p.toUpperCase(), r = i >> 3, c = i & 7;
    let v = CHV[t];
    if (t === "p") v += (w ? 6 - r : r - 1)
* 8; else if (t !== "k" && t !== "r") v +=
10 - (Math.abs(3.5 - r) + Math.abs(3.5 -
c)) * 3;
    s += w ? v : -v;
  }
  return s;
}
function chSearch(b, d, a, be, w) {
  if (d === 0) return (w ? 1 : -1) *
chEval(b);
  const ms = chGen(b, w); if (!ms.length)
return 0;
  const val = m => b[m[1]] ?
CHV[b[m[1]].toLowerCase()] : 0;
  ms.sort((x, y) => val(y) - val(x));
  let best = -1e9;
  for (const m of ms) {
    if (b[m[1]] && b[m[1]].toLowerCase()
=== "k") return 1e5 + d;
    const v = -chSearch(chDo(b, m), d - 1,
-be, -a, !w);
    if (v > best) best = v; if (best > a) a
= best; if (a >= be) break;
  }
}

```

```

    return best;
}
function chAI(b, lv) {
    const ms = chLegal(b, false), depth = [2,
3, 4][lv]; let best = null, bs = -1e9;
    for (const m of ms) {
        const v = -chSearch(chDo(b, m), depth -
1, -1e9, 1e9, true) + Math.random() * 3;
        if (v > bs) { bs = v; best = m; }
    }
    return best;
}
function chStatus(b, w) { const n =
chLegal(b, w).length, ck = chInCheck(b, w);
return n ? (ck ? "check" : "ok") : (ck ?
"mate" : "stale"); }

/* ----- Échecs : 3D ----- */
let ch = null;
function chPiece(p, idx) {
    const t = p.toLowerCase(), w = p ===
p.toUpperCase();
    const mat = new
THREE.MeshStandardMaterial({ color: w ?
0xf3efe6 : 0x2b2b33, metalness: 0.2,
roughness: 0.5 });
    const g = new THREE.Group();
    const add = (geo, y) => { const x = new
THREE.Mesh(geo, mat); x.position.y = y;
g.add(x); return x; };
    add(new THREE.CylinderGeometry(0.34, 0.4,
0.16, 20), 0.18);
    if (t === "p") { add(new
THREE.CylinderGeometry(0.14, 0.26, 0.4,

```

```

16), 0.46); add(new
THREE.SphereGeometry(0.2, 16, 12), 0.78); }
    else if (t === "r") { add(new
THREE.CylinderGeometry(0.25, 0.3, 0.6, 16),
0.55); add(new THREE.CylinderGeometry(0.33,
0.27, 0.2, 16), 0.95); }
    else if (t === "n") { add(new
THREE.CylinderGeometry(0.2, 0.3, 0.5, 16),
0.5); const h = add(new
THREE.BoxGeometry(0.28, 0.5, 0.5), 0.95);
h.position.z = 0.08; h.rotation.x = -0.4; }
    else if (t === "b") { add(new
THREE.ConeGeometry(0.28, 0.8, 18), 0.62);
add(new THREE.SphereGeometry(0.15, 14, 10),
1.1); }
    else if (t === "q") { add(new
THREE.CylinderGeometry(0.18, 0.32, 0.9,
18), 0.66); add(new
THREE.SphereGeometry(0.24, 16, 12), 1.3); }
    else { add(new
THREE.CylinderGeometry(0.2, 0.32, 1, 18),
0.7); add(new THREE.BoxGeometry(0.1, 0.4,
0.1), 1.4); add(new THREE.BoxGeometry(0.3,
0.1, 0.1), 1.4); }
    g.traverse(x => { x.userData.i = idx; });
    g.position.set((idx & 7) - 3.5, 0.1, (idx
>> 3) - 3.5);
    return g;
}
function chDraw() {
    const o = ch.o;
    ch.sq.forEach((s, i) => {
        s.material.emissive.setHex(i === ch.sel ?
0x3355ff : ch.moves.includes(i) ? 0x1f9d4a

```

```

: 0x000000); });
    if (ch.pg) o.scene.remove(ch.pg);
    ch.pg = new THREE.Group();
    ch.b.forEach((p, i) => { if (p)
ch.pg.add(chPiece(p, i)); });
    o.scene.add(ch.pg);
    $("chMsg").textContent = ch.msg;
}
function startChess() {
    pickLevel("♙ Échecs 3D", lv =>
loadThree(() => {
        ensurePage("chessPage", `<div
class="quizbox"><h2>♙ Échecs 3D</h2><div
id="chHost"></div><p id="chMsg"
style="font-weight:bold;min-height:24px">
</p><p class="muted">Tu joues les blancs.
Touche une pièce puis une case verte.</p>
<button class="primary"
onclick="startWithBet('startChess')">↺
Nouvelle partie</button> <button
class="primary"
onclick="showPage('games')">←
Jeux</button></div>`);
        showPage("chessPage");
        const o = mk3d($("chHost"), 0.95);
        o.cam.position.set(0, 9.5, 8.5);
o.cam.lookAt(0, 0, 0.6);
        const sq = [];
        for (let i = 0; i < 64; i++) {
            const dark = ((i >> 3) + (i & 7)) % 2
=== 1;
            const m = new THREE.Mesh(new
THREE.BoxGeometry(1, 0.2, 1), new
THREE.MeshStandardMaterial({ color: dark ?

```

```

0x6b4a2f : 0xe8d3a8 }));
    m.position.set((i & 7) - 3.5, 0, (i
>> 3) - 3.5); m.userData.i = i;
o.scene.add(m); sq.push(m);
    }
    const frame = new THREE.Mesh(new
THREE.BoxGeometry(8.6, 0.16, 8.6), new
THREE.MeshStandardMaterial({ color:
0x2b1a0e }));
    frame.position.y = -0.08;
o.scene.add(frame);
    ch = { o, sq, b: chInit(), sel: -1,
moves: [], lv, over: false, turn: true,
msg: "À toi de jouer (blancs).", pg: null
};
    chDraw();

o.r.domElement.addEventListener("pointerdown", e => {
    const objs = sq.concat(ch.pg ?
[ch.pg] : []);
    const h = pick3d(o, e, objs).find(x
=> x.object.userData.i !== undefined);
    if (h) chClick(h.object.userData.i);
    });
});
}
function chClick(i) {
    if (!ch || ch.over || !ch.turn) return;
    const p = ch.b[i];
    if (ch.sel >= 0 && ch.moves.includes(i))
    {
        ch.b = chDo(ch.b, [ch.sel, i]); ch.sel
= -1; ch.moves = []; ch.turn = false;

```

```

    const st = chStatus(ch.b, false);
    if (st === "mate") return chEnd("🏆
Échec et mat ! Tu gagnes !", Math.round(32
* MULT[ch.lv]));
    if (st === "stale") return chEnd("🙄
Pat : match nul.", 10);
    ch.msg = st === "check" ? "Échec au roi
noir ! L'ordinateur réfléchit..." :
"L'ordinateur réfléchit...";
    chDraw();
    const g = ch;
    setTimeout(() => {
        if (ch !== g || g.over) return;
        const m = chAI(g.b, g.lv);
        g.b = chDo(g.b, m); g.turn = true;
        const s2 = chStatus(g.b, true);
        if (s2 === "mate") return chEnd("💥
Échec et mat : l'ordinateur gagne.", 0);
        if (s2 === "stale") return chEnd("🙄
Pat : match nul.", 10);
        g.msg = s2 === "check" ? "⚠️ Ton roi
est en échec !" : "À toi de jouer.";
        chDraw();
    }, 450);
    } else if (p && p === p.toUpperCase()) {
        ch.sel = i; ch.moves = chLegal(ch.b,
true).filter(m => m[0] === i).map(m =>
m[1]); chDraw();
    } else { ch.sel = -1; ch.moves = [];
chDraw(); }
}
function chEnd(msg, pts) {
    ch.over = true; ch.msg = msg + (pts ? `
+${pts} points` : "");

```

```
    if (pts) addPoints(pts);
    chDraw();
    gameOver(pts > 10 ? true : pts === 10 ?
"draw" : false, msg);
}

/* ----- Démarrage ----- */
// boutons du menu (Accueil, Jeux,
Classement, Récompenses, Admin)
document.querySelectorAll("nav button[data-
page]").forEach(b => {
    b.addEventListener("click", () =>
showPage(b.getAttribute("data-page")));
});
render();
/* ===== FIN BLOC 5 ===== */
```